UR THE INNOVATIVE UNIVERSITY	Challeneg4	Ing4-CYB/CCV	S7

Course Outcomes (Syllabus) avec le pourcentage		
CO	**Exercice**	**%**
CO1: Understand the fundamentals of PP and PA	Ex1	30%
CO2: Apply knowledge and develop hands-on skills in developing applications	Ex2	30%
CO3: Evaluate parallel code efficiency for HPC tuning and optimization	Ex3	40%

Nom[1] :.....**Prénom**:.....

Suppose we are given an integer to factor, that is, to decompose it into a product of prime numbers. Fermat's factorization method is based on the identity, where x and y are integers.

$$N = x^2 - y^2 = (x - y)(x + y),$$

We begin with:

$$x = \left\lceil \sqrt{N} \right\rceil = \lceil \sqrt{35} \rceil = \lceil 5.91 \rceil = 6.$$

Then compute $y^2 = x^2 - N$, $y^2 = 6^2 - 35 = 36 - 35 = 1$, since 1 is a perfect square, we have $y = 1$. We can now compute the factors of N, using the identity $(x - y)(x + y)$: $x - y = 6 - 1 = 5$, $x + y = 6 + 1 = 7$. So, the factorization of 35 is: $35 = 5 \times 7$.
Fermat's factorization is easily parallelizable because each candidate value of x can be tested independently, allowing multiple processes or threads to search different values of x simultaneously without communication. The parallel Fermat factorization algorithm can be given as follows:

[1] Tous documents autorisés

Algorithm 1: Parallel Fermat's Factorization

Data: Integer N to factor, number of threads/processes T
Result: Factors p and q such that $N = p \cdot q$
foreach *thread* t *in* $0..T-1$ ***in parallel*** **do**
  $x \leftarrow \lceil \sqrt{N} \rceil + t$;
  while $x^2 - N$ *is not a perfect square* **do**
    $x \leftarrow x + T$;
  end
  $y \leftarrow \sqrt{x^2 - N}$;
  store the result if it has not already been found by another thread;
end
return $(x - y, x + y)$

a. (CO1, CO2, 12pt) Complete the CUDA parallel program (fermat.cu).

b. (CO3, 8pt) Evaluate the performance of the Fermat factorization algorithm using the following numbers. For each test case, measure the execution time on both the CPU and GPU using different configurations (e.g., the number of blocks and the number of threads per block). Analyze how the execution time and speedup vary with both the size of (N) and the considered configurations.

Table 1: Test cases for scalability and performance

Test	p	q	$N = p \times q$
1	1 000 003	1 000 033	1 000 036 000 099
2	2 000 003	2 000 033	4 000 072 000 099
3	5 000 009	5 000 029	25 000 190 000 261
4	10 000 019	10 000 079	100 000 980 001 501
5	20 000 003	20 000 033	400 000 720 000 099
6	50 000 009	50 000 029	2 500 001 900 000 261
7	100 000 007	100 000 037	10 000 004 400 000 259
8	200 000 003	200 000 033	40 000 007 200 000 099
9	500 000 009	500 000 029	250 000 019 000 000 261
10	1 000 000 007	1 000 000 033	1 000 000 040 000 000 231